

Lab Report

23CSE212 – Principles of Functional Languages

Criteria	Excellent	Good	Poor
Timely Submission			
Correctness of lab assignment			
Total Marks			
Signed By Lab Instructor			

Lab Session No: 5 Date:14 /03 /2025	
Question 1 : Generate a list of squares of numbers from 1 to 5. CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.	
Code	Testcases (Input & Output)
<pre>square :: Num a => [a] -> [a] square [] = [] square (x:xs) = x*x : square xs</pre>	<pre>ghci> square [1..5] [1,4,9,16,25]</pre>

Question 2 : Generate a list of even numbers from m to n . CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.	
Code	Testcases (Input & Output)
<pre>evennums :: Int -> Int -> [Int] evennums m n = [x x <- [m..n], even x]</pre>	<pre>ghci> evennums 1 10 [2,4,6,8,10] ghci> evennums 0 2 [0,2]</pre>

Lab Report

23CSE212 – Principles of Functional Languages

Question 3 : Generate a list of first n odd numbers. CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.	
Code	Testcases (Input & Output)
<pre>oddnums :: Int -> [Int] oddnums n = [x x <- [0..n], odd x]</pre>	<pre>ghci> oddnums 10 [1,3,5,7,9] ghci> oddnums 25 [1,3,5,7,9,11,13,15,17,19,21,23,25]</pre>

Question 4 : Generate a list of numbers from m to n that are divisible by 3. CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.	
Code	Testcases (Input & Output)
<pre>divisibleby3 :: Int -> Int -> [Int] divisibleby3 m n = [x x <- [m..n], x `mod` 3 == 0]</pre>	<pre>ghci> divisibleby3 1 15 [3,6,9,12,15] ghci> divisibleby3 1 20 [3,6,9,12,15,18]</pre>

Question 5 : Generate all ordered pairs (x, y) from x is from xs and y is from ys lists. (xs=[1,2,3],ys=[4,5]; Output= [(1,4), (1,5), (2,4), (2,5), (3,4), (3,5)]) CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.	
Code	Testcases (Input & Output)
<pre>orderedpairs :: [Int] -> [Int] -> [(Int, Int)] orderedpairs list1 list2 = [(x, y) x <- list1, y <- list2]</pre>	<pre>ghci> orderedpairs [1..3] [4..5] [(1,4),(1,5),(2,4),(2,5),(3,4),(3,5)] ghci> orderedpairs [1..3] [4..7] [(1,4),(1,5),(1,6),(1,7),(2,4),(2,5),(2,6),(2,7),(3,4),(3,5),(3,6),(3,7)]</pre>

Lab Report

23CSE212 – Principles of Functional Languages

Question 6 : Generate all pairs (x, y) where x is from xs and y is from ys but only include pairs where x + y is even. (*xs*=[1,2,3],*ys*= [1,2,3]; *Output*= [(1,1), (1,3), (2,2), (3,1), (3,3)])

CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
<pre>evenpairs :: [Int] -> [Int] -> [(Int, Int)] evenpairs list1 list2 = [(x, y) x <- list1, y <- list2, even (x + y)]</pre>	<pre>ghci> evenpairs [1,2,3] [1,2,3] [(1,1),(1,3),(2,2),(3,1),(3,3)] ghci> evenpairs [1,2,3,4] [1,2,3,4,5] [(1,1),(1,3),(1,5),(2,2),(2,4),(3,1),(3,3),(3,5),(4,2),(4,4)]</pre>

Question 7 : Flatten a list of lists. [*Input*: [[1,2], [3,4], [5,6]], *Output*= [1,2,3,4,5,6])

CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
<pre>flatlist :: [[Int]] -> [Int] flatlist [] = [] flatlist (x:xs) = x ++ flatlist xs</pre>	<pre>ghci> flatlist [[1,2],[3,4],[5,6]] [1,2,3,4,5,6]</pre>

Question 8 : Find all Pythagorean triples where a , b, and c are from m to n , and $a^2 + b^2 = c^2$. (*m*=1 and *n*=20, *Output*= [(3,4,5), (4,3,5), (5,12,13), (6,8,10), (8,6,10), (9,12,15), (12,5,13), (12,9,15), (15,8,17), (20,15,25)])

CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
<pre>pythagorepairs :: Int -> Int -> [(Int, Int, Int)] pythagorepairs m n = [(a, b, c) a <- [m..n], b <- [m..n], c <- [m..n], (a*a + b*b) == c*c]</pre>	<pre>ghci> pythagorepairs 1 20 [(3,4,5),(4,3,5),(5,12,13),(6,8,10),(8,6,10), (8,15,17),(9,12,15),(12,5,13),(12,9,15), (12,16,20),(15,8,17),(16,12,20)] ghci> pythagorepairs 1 15 [(3,4,5),(4,3,5),(5,12,13),(6,8,10),(8,6,10), (9,12,15),(12,5,13),(12,9,15)]</pre>

Lab Report

23CSE212 – Principles of Functional Languages

Question 9: Generate a list of all two-digit numbers where the first digit is even, and the second digit is odd. (Output: [10, 12, 14, 16, 18, 30, 32, 34, 36, 38, 50, 52, 54, 56, 58, 70, 72, 74, 76, 78, 90, 92, 94, 96, 98])

CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
<pre>twodigit::[Int] twodigit = [x*10 + y x <- [1..9], y <- [0..9], odd x, even y, x*10 + y >= 10, x*10 + y <= 99]</pre>	<pre>twodigit [10,12,14,16,18,30,32,34,36,38,50,52,54,56,58 ,70,72,74,76,78,90,92,94,96,98]</pre>

Question 10 :Extract all digits from the string. (Input: "Haske1112341s567Fun890!", Output: "1234567890").

CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
<pre>extractDigits :: String -> String extractDigits s = [c c <- s, c >= '0', c <= '9']</pre>	<pre>ghci> extractDigits "Haskell1112234" "1112234" ghci> extractDigits "hello" ""</pre>

Question 11 : Generate a list of sums of each sublist in xs list. (Input: [[1,2,3], [4,5], [6,7,8,9]], Output: [6, 9, 30])

CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
<pre>sublistsum :: [[Int]] -> [Int] sublistsum list = [sum x x <- list]</pre>	<pre>ghci> sublistsum [[1,2,3],[4,5],[6,7,8,9]] [6,9,30]</pre>

Lab Report

23CSE212 – Principles of Functional Languages

Question 12 : Generate all triplets (x, y, z) where x, y, and z are from xs and $x+y+z=s$. (Input: [1..5], s=10, Output: [(1,4,5), (1,5,4), (2,3,5), (2,4,4), (2,5,3), (3,2,5), (3,3,4), (3,4,3), (3,5,2), (4, 1,5), (4,2,4), (4,3,3), (4,4,2), (4,5,1), (5, 1,4), (5,2,3), (5,3,2), (5,4, 1)])

CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
<pre>triplets :: [Int] -> Int -> [(Int, Int, Int)] triplets list s = [(x, y, z) x <- list, y <- list, z <- list, (x + y + z) == s]</pre>	<pre>ghci> triplets [1..5] 10 [(1,4,5),(1,5,4),(2,3,5),(2,4,4),(2,5,3),(3,2,5),(3,3,4), (3,4,3),(3,5,2),(4,1,5),(4,2,4),(4,3,3),(4,4,2),(4,5,1), (5,1,4),(5,2,3),(5,3,2),(5,4,1)]</pre>

Question 13 : Write a function *divisors* n that returns all divisors of n using list comprehension.

Testcase: *divisors* 12 Output: [1, 2, 3, 4, 6, 12]

CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
<pre>divisors :: Int -> [Int] divisors n = [x x <- [1..n], n `mod` x == 0]</pre>	<pre>ghci> divisors 12 [1,2,3,4,6,12] ghci> divisors 15 [1,3,5,15]</pre>

Question 14 : Write a function *factors* n that returns all prime factors of n using list comprehension.

Testcase: *factors* 28 Output: [2, 7]

CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
<pre>isPrime :: Int -> Bool isPrime n n <= 1 = False otherwise = null [x x <- [2..floor (sqrt (fromIntegral n))], n `mod` x == 0] factors n = [x x <- [2..n], n `mod` x == 0, isPrime x]</pre>	<pre>ghci> factors 28 [2,7] ghci> factors 45 [3,5]</pre>

Lab Report

23CSE212 – Principles of Functional Languages

Question 15 : Write a function *isPrime* *n* that checks if *n* is prime using list comprehension.

Testcases: Outputs:

isPrime 7 *True*

isPrime 10 *False*

CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
<pre>isPrime :: Int -> Bool isPrime n n <= 1 = False otherwise = null [x x <- [2..floor (sqrt (fromIntegral n))], n `mod` x == 0]</pre>	<pre>ghci> isPrime 7 True ghci> isPrime 10 False</pre>

Question 16 : Write a function *extractVowels* that takes a list of words and returns a list of only the vowels from each word.

Testcase: extractVowels ["Haskell", "Functional", "Magic"]

Output: ["ae","uaioa","ai"]

CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
<pre>extractvowels :: [String] -> [String] extractvowels list = [[y y <- x, toLower y `elem` "aeiou"] x <- list]</pre>	<pre>ghci> extractvowels ["Haske","Functional","Magic"] ["ae","uioa","ai"] ghci> extractvowels ["abcd","efght"] ["a","e"]</pre>

Question 17 : Write a function *cartesianProduct* that generates a list of all pairs (x, y) where x is from [1..m] and y is from [1..n]

Testcase: cartesianProduct 2 3

Output: [(1,1),(1,2),(1,3),(2,1),(2,2),(2,3)]

CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
<pre>cartesianProduct :: Int -> Int -> [(Int, Int)] cartesianProduct m n = [(x, y) x <- [1..m], y <- [1..n]]</pre>	<pre>ghci> cartesianProduct 2 3 [(1,1),(1,2),(1,3),(2,1),(2,2),(2,3)] ghci> cartesianProduct 3 6 [(1,1),(1,2),(1,3),(1,4),(1,5),(1,6),(2,1),(2,2), (2,3),(2,4),(2,5),(2,6),(3,1),(3,2),(3,3),(3,4), (3,5),(3,6)]</pre>

Lab Report

23CSE212 – Principles of Functional Languages

Question 18 : Write a function <i>multiplicationTable</i> that takes an integer n and returns a list of its multiplication table up to 10. <i>Testcase: multiplicationTable 5</i> <i>Output:</i> <i>[5,10,15,20,25,30,35,40,45,50]</i> CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.	
Code	Testcases (Input & Output)
<pre>multiplicationtable :: Int -> [Int] multiplicationtable n = [x * n x <- [1..10]]</pre>	<pre>multiplicationtable 5 [5,10,15,20,25,30,35,40,45,50] ghci> multiplicationtable 15 [15,30,45,60,75,90,105,120,135,150]</pre>

Question 19. Write a function <i>triangularNumbers</i> that generates the first n triangular numbers. (A triangular number is the sum of all integers up to a given number: 1, 3, 6, 10, 15, ...) <i>Testcase:</i> <i>triangularNumbers 5</i> <i>Output: [1,3,6,10,15]</i> CO2: Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.	
Code	Testcases (Input & Output)
<pre>triangularnumbers :: Int -> [Int] triangularnumbers n = [(x * (x + 1)) `div` 2 x <- [1..n]]</pre>	<pre>ghci> triangularnumbers 5 [1,3,6,10,15] ghci> triangularnumbers 10 [1,3,6,10,15,21,28,36,45,55]</pre>

Question 20. Write a function <i>commonElements</i> that returns the common elements between two lists. <i>Testcase: commonElements [1, 2, 3, 4, 5] [3, 4, 5, 6, 7]</i> <i>Output: [3,4,5]</i> CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.	
Code	Testcases (Input & Output)
<pre>commonelements :: [Int] -> [Int] -> [Int] commonelements list1 list2 = [x x <- list1, x `elem` list2]</pre>	<pre>ghci> commonelements [1,2,3,4,5] [3,4,5,6,7] [3,4,5] ghci> commonelements [1,2,3,4,5] [6..10] []</pre>

Lab Report

23CSE212 – Principles of Functional Languages

Question 21. Write a function <i>sumPairs</i> that takes two lists and returns a list of sums of corresponding elements using <i>zip</i> and list comprehension. Testcase: <i>sumPairs [1, 2, 3] [4, 5, 6]</i> Output: <i>[5, 7, 9]</i> CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.	
Code	Testcases (Input & Output)
<pre>sumpairs :: [Int] -> [Int] -> [Int] sumpairs list1 list2 = [x + y (x, y) <- zip list1 list2]</pre>	<pre>ghci> sumpairs [1,2,3] [4,5,6] [5,7,9] ghci> sumpairs [1,2,3] [4,5,6,7] [5,7,9]</pre>

Question 23. Write a function <i>pairConsecutive</i> that takes a list and returns a list of consecutive element pairs using <i>zip</i> and list comprehension. Testcase: <i>pairConsecutive [1, 2, 3, 4, 5]</i> Output: <i>[(1,2), (2,3), (3,4), (4,5)]</i> CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.	
Code	Testcases (Input & Output)
<pre>pairconsecutive :: [Int] -> [(Int, Int)] pairconsecutive list = zip list (tail list)</pre>	<pre>ghci> pairconsecutive [1..5] [(1,2),(2,3),(3,4),(4,5)] ghci> pairconsecutive [1..10] [(1,2),(2,3),(3,4),(4,5),(5,6),(6,7),(7,8), (8,9),(9,10)]</pre>

Question 24. Write a function <i>differences</i> that computes the absolute differences between consecutive elements using <i>zip</i> and list comprehension.Testcase: <i>differences [10, 20, 15, 30, 25]</i> Output• <i>[10, 5, 15, 5]</i> CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.	
Code	Testcases (Input & Output)
<pre>differences :: Num a => [a] -> [a] differences xs = [abs (x - y) (x, y) <- zip xs (tail xs)]</pre>	<pre>ghci> differences [10,20,15,30,25] [10,5,15,5] ghci> differences [50,55,49,38,63] [5,6,11,25]</pre>

Lab Report

23CSE212 – Principles of Functional Languages

Question 25. Write a function *reversePairs* that takes a list and returns a list of pairs, where each pair consists of an element and its counterpart from the reversed list.

Testcase: *reversePairs* [1, 2, 3, 4, 5] *Output:* [(1,5), (2,4), (3,3), (4,2), (5,1)]

CO2 : Develop Haskell programs to solve basic programming problems based on type classes,function definitions, higher-order functions, and list processing.

Code	Testcases (Input & Output)
<pre>reversePairs :: [Int] -> [(Int, Int)] reversePairs xs = zip xs (reverse xs)</pre>	<pre>ghci> reversePairs [1..5] [(1,5),(2,4),(3,3),(4,2),(5,1)] ghci> reversePairs [1..10] [(1,10),(2,9),(3,8),(4,7),(5,6),(6,5),(7,4), (8,3),(9,2),(10,1)]</pre>